

# CSC3002F: Scheduling Algorithms

Paul Kabulu - KBLPAU002

May 2026

# 1 Methods

## 1.1 Experimental Setup

Four scheduling algorithms were tested: FCFS, SJF, Priority, and MLFQ. Each ran 80 simulations using patron counts of 10, 20, 30, and 50, with 5 different random seeds per configuration. The shell script using a grid search to run the program with all possible args combinations (4 patron counts  $\times$  5 seeds  $\times$  4 schedulers).

## 1.2 Metrics and Data Collection

For each order, the script recorded arrival time, service start time, completion time, and queue level. I then calculated waiting time, turnaround time, throughput, and per-patron total wait using a python script. A shell script automated the simulations, with results written to CSV files by the modified `recordCompletedOrder()` method in `Barman.java`.

## 1.3 Analysis & Visualisation

I used Python to combine the 80 data files and pull out the key descriptive statistics: the mean, median, standard deviation, and maximum wait times. To visually compare how the algorithms performed under different conditions, I used Matplotlib to generate nine plots.

## 1.4 Experimental Controls

The simulation was used without changing core scheduling logic. Fixed seeds enable reproducibility. Because I used fixed random seeds, the patron arrival orders and drink workloads were identical across all four algorithms. This ensured that any differences in performance were due to the scheduling algorithms themselves.

# 2 Results

## 2.1 Performance at Different Workload Sizes

SJF achieved the lowest average waiting times across all configurations. At 50 patrons, SJF averaged  $\sim 1000$  ms compared to FCFS ( $\sim 1500$  ms) and MLFQ ( $\sim 1600$  ms). This matches the expected theoretical behaviour of SJF, since shorter jobs are prioritised and completed faster, which helps reduce queue congestion overall. This results in the prevention of a long queue buildup. FCFS had a moderate and predictable performance. Conversely, MLFQ performed slightly worse than expected, possibly because the 4000 ms aging threshold was set too high.

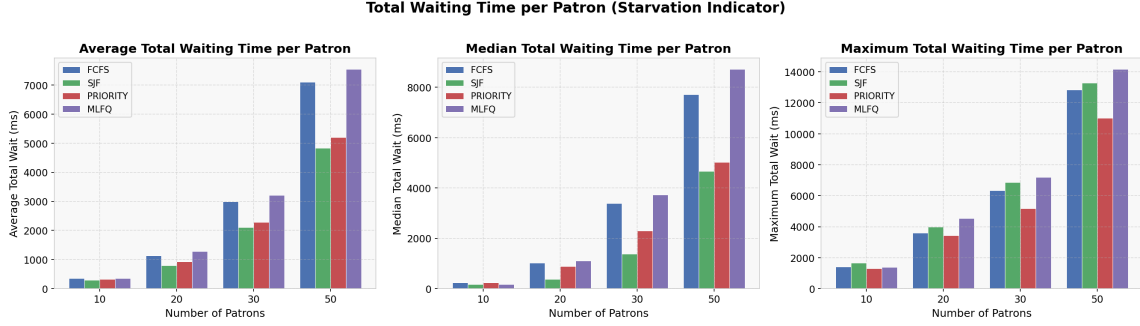


Figure 1: Total Wait Per Patron

The median values, however, show a different pattern. SJF had a median waiting time of only  $\sim 100$  ms at 50 patrons, which is much lower than its average. This suggests that short jobs were completed very quickly, while longer jobs experienced much higher waiting times that increased the overall average. FCFS had median and mean values that were much closer together, which might mean more stable and consistent waiting times across different job lengths.

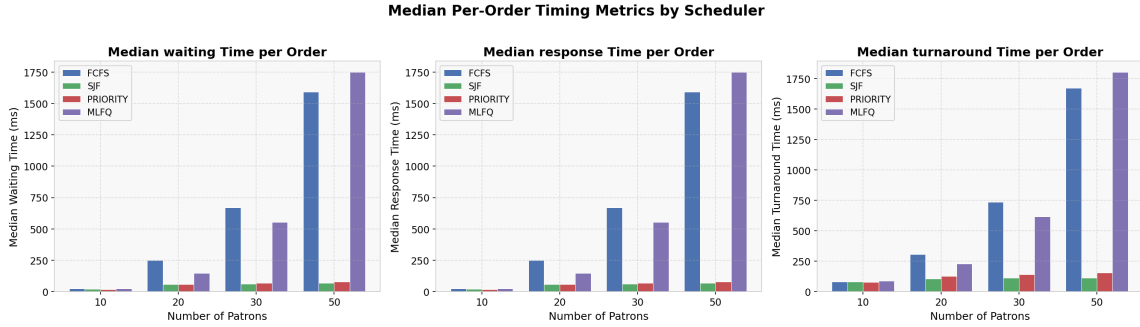


Figure 2: Total Wait Per Patron

## 2.2 Throughput

All the schedulers achieved almost the same throughput, around  $\sim 10.8$ – $15.3$  orders/sec, which shows that the scheduling policy itself doesn't really affect the total amount of work completed in non-preemptive algorithms. This makes sense because the barman is still doing the same amount of work regardless of the order in which jobs are processed. The difference between the algorithms lies in fairness rather than the throughput.

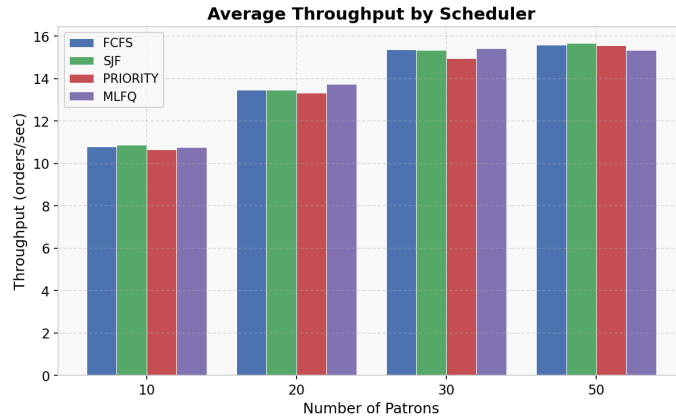


Figure 3: Average Throughput by Scheduler

## 2.3 Predictability

Looking at the standard deviation of waiting times under high load (50 patrons) gives a better idea of how predictable each scheduling algorithm is:

- FCFS:  $\sim 800$  ms
- MLFQ:  $\sim 1100$  ms
- SJF:  $\sim 2200$  ms
- Priority:  $\sim 2300$  ms

We know that the lower the variance, the more predictable a dataset is because most data points are centred close to the mean. FCFS and MLFQ had much lower variance, which means customers would have a better idea of how long they would have to wait based on the queue size. On the other hand, SJF and Priority showed very high variance in waiting times. This means some patrons could get served very quickly, while others might end up waiting much longer depending on the type or priority of their job.

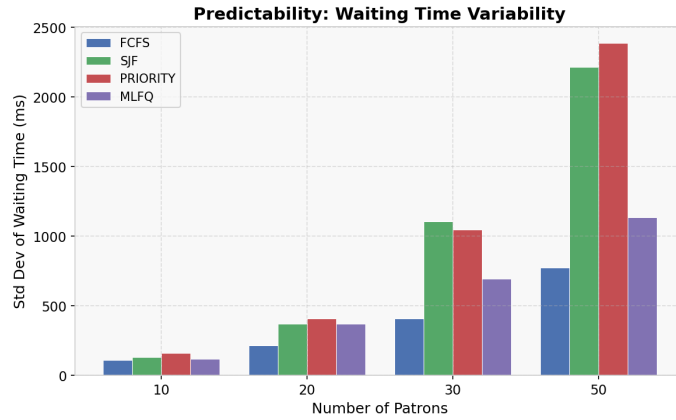


Figure 4: Waiting Time Variance

## 2.4 Starvation Risk

Maximum waiting times show the worst-case scenario:

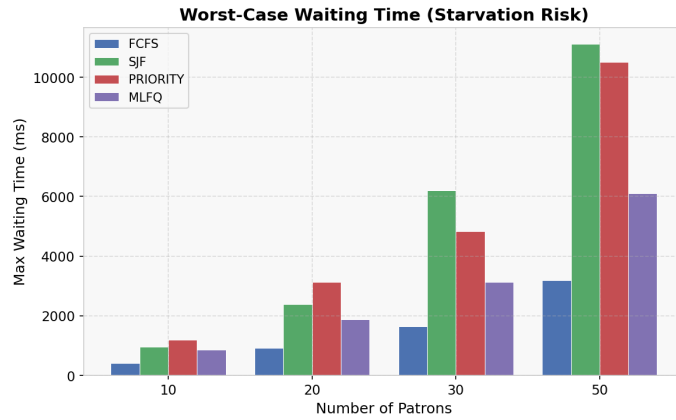


Figure 5: Waiting Time Variance

This shows the issue of starvation that SJF scheduling suffers from. In MLFQ, aging helps reduce this issue because once a patron has waited long enough, their order is moved to a higher-priority queue. This prevents extremely long delays and keeps the maximum waiting time closer to 6 seconds. FCFS does not experience starvation because orders are always handled in the exact order they arrive.

## 2.5 Fairness

Analysing cumulative waiting time across each patron's entire stay shows very clear differences between the scheduling algorithms:

**Fairness: Total Waiting Time per Patron**

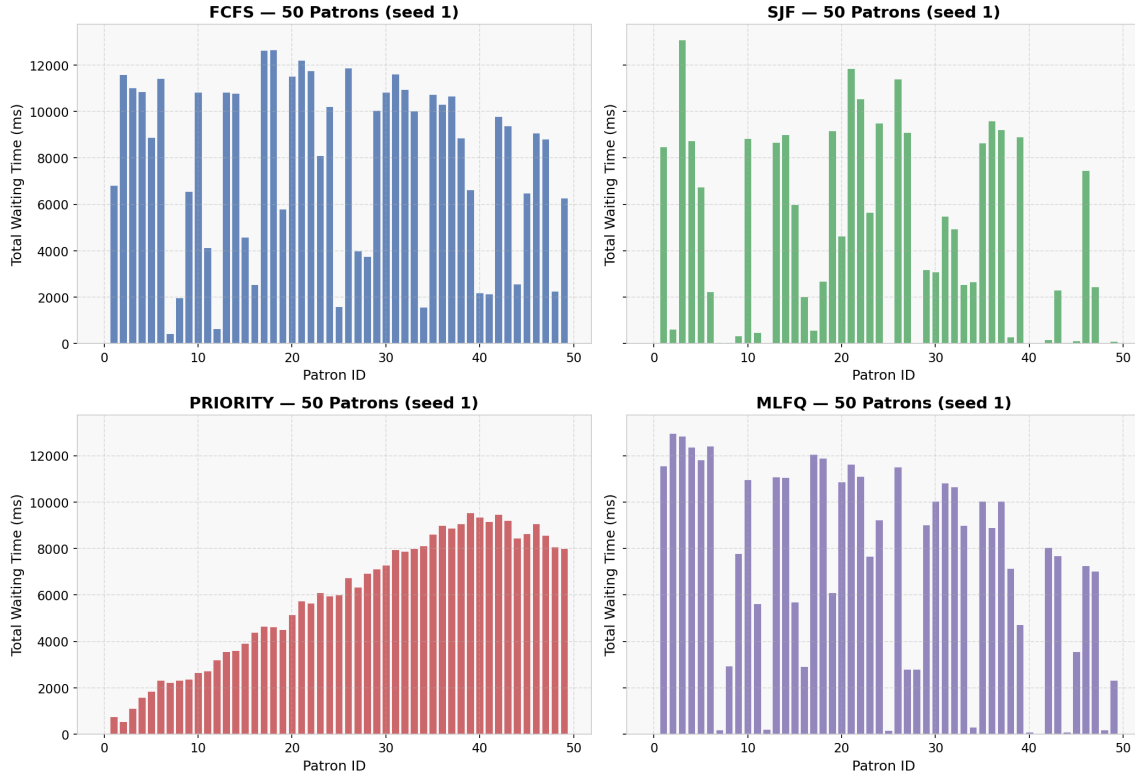


Figure 6: Total Waiting Time

- **FCFS** produces a more uniform looking distribution, with total waiting times in the  $\sim 6000$ – $12,000$  ms range. It treats all patrons the same based on the time they arrive, so there's no advantage or lack thereof other than queue position.
- **Priority** scheduling shows an increasing/linear pattern. Early patrons have much lower total waiting time, around  $\sim 1500$  ms, while later ones can reach  $\sim 9000$  ms. This shows a clear advantage that the priority brings, where waiting time is dependant on the priority.
- **SJF** creates an unbalanced distribution. Customers with simple orders (shorter jobs) experience low wait times ( $\sim 2000$  ms), but those with more complex orders (longer jobs) can wait much longer, up to  $\sim 13,000$  ms.

This means performance depends on the order's complexit rather than being fair towards all patrons.

- **MLFQ** sits in the middle, showing a more balanced distribution than SJF. There is still some variation depending on order size, but aging helps prevent the extreme imbalance seen in SJF by making sure that long waiting patrons have higher priority.

## 2.6 Distribution

Box plots show a consistent pattern across workloads. At 10 patrons, all algorithms perform similarly with very little variation because the system is lightly loaded. As the load increases to 30–50 patrons, differences become clearer.

SJF shows the most variance, with extreme outliers under heavy load, meaning waiting times become inconsistent. MLFQ stays much more balanced, with a tighter distribution (clusters closer to the mean than SJF and priority) even as load increases. At 50 patrons, aging helps prevent very large delays in both SJF and Priority scheduling, keeping results more controlled overall.

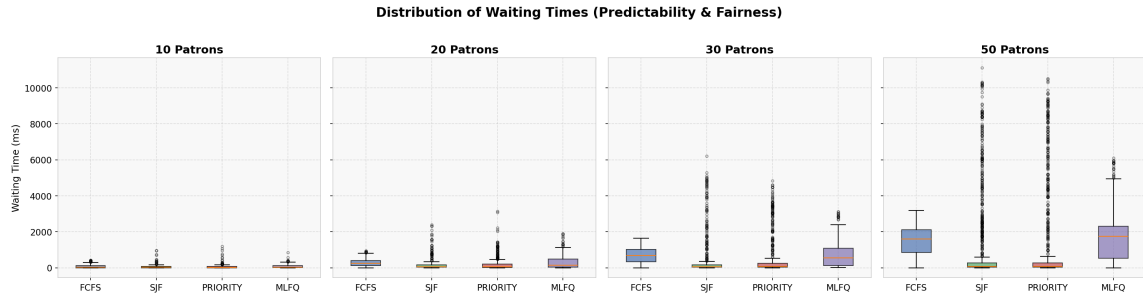


Figure 7: Waiting Time Distribution

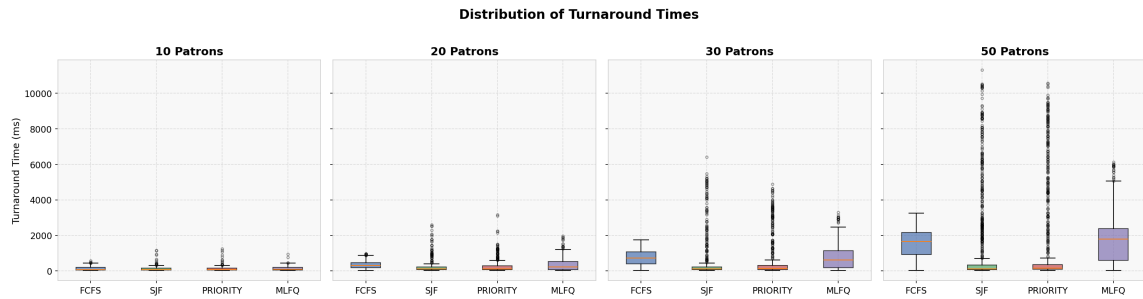


Figure 8: Turn around time Distribution

### 3 Conclusion

**MLFQ** provides the best overall balance for Allegra's bar.

**SJF** gives the lowest average wait ( $\sim 513$  ms), but it is unfair in practice. Its long maximum wait (up to 11 seconds) means larger orders can be heavily delayed, which is not the best design choice.

**FCFS** is the fairest and most predictable option, but it has slower overall performance compared to SJF and MLFQ.

**MIFQ** performs closest to SJF in average wait ( $\sim 765$  ms) while avoiding very long delays. Its maximum wait (6000 ms) is much lower than SJF, and variance is somewhat decent due to aging which improves fairness by preventing long waits.

**Priority scheduling** does not have any real advantage, due to it being both unfair and inconsistent majority of the time.

### 4 AI Declaration

The bash script was written by Anthropic's Claude AI. I also received help from Claude AI in plotting the figures. It helped me load and combine all the data using Pandas. It also helped with some of the calculations in the plot related methods.

Generative AI was not used to fabricate the data.

All reported metrics, figures, and conclusions are derived directly from the 80 experimental runs and corresponding CSV outputs.